Parallel Computing and Symmetrical Concurrency Architectures

To maximize pipeline throughput under heavy multimodal payloads, the system implements three distinct concurrent execution profiles. Symmetrically, these correspond to stateless single-turn bulk tasks, stateful multi-turn agentic processes, and hybrid decoupled indexing pipelines:

- **Standard Parallel Steps:** Stateless, single-turn bulk operations (such as asset captioning and asset layer proposals). Symmetrically, these can safely share a standard globally pooled connection pool because each socket is quickly acquired for a single roundtrip and released immediately without keep-alive states.
- **Agentic Loops (Socket-Separated Parallel Computing):** Stateful, multi-turn conversational agents (such as the macro-discovery loop, the micro-subdivision loop, and the decoupled verification loop). Symmetrically, these require private connection pools and file descriptors exclusively bound to each thread's ID via thread-local caching to prevent socket multiplexing collisions.
- **Hybrid Decoupled Indexing (Parallel Batch Vectorization):** High-performance batch compilation channels (such as captions indexing, image crops indexing, and prose text chunks indexing). Symmetrically, these decouple network-bound operations (such as calling translation APIs concurrently via thread pools) from local compute-bound operations (such as local text and joint vision model vectorization).

Symmetrically, visual sequence flowcharts representing all three profiles are depicted inside Standard Parallel Computing Profile, Socket-Separated Agentic Loop Profile, and Hybrid Parallel Computing Profile.

Concurrency and Concurrency Profile Matrix

Five pipeline steps run several workers in parallel. The other steps run sequentially. Defaults below are sourced from the agent and worker configuration modules inside the orchestration directory:

Pipeline Step Name	Concurrency Profile Class	What runs in parallel	Work unit	CLI flag	Default
`macro-images`	**Agentic Loops (Socket-Separated)**	the macro cropping agent	one session per (doc, page)	`--workers N`	8
`micro-images`	**Agentic Loops (Socket-Separated)**	the micro cropping agent	one session per committed macro	`--workers N`	8
`verify-drafts`	**Agentic Loops (Socket-Separated)**	the programmatic verification loop	one thread per draft	`--workers N`	8
`caption`	**Standard Parallel Steps**	the captioner	one call per approved region	`--workers N`	4
`asset-layers`	**Standard Parallel Steps**	the layer-assigner	one call per asset	`--workers N`	4
`build-index`	**Hybrid Decoupled Indexing**	the translator (network threads) + batched local vectorizers (C++ matrix passes)	one call per chunk/asset (translation) + batched runs (embeddings)	`--workers N`	8

To run any of these strictly one item at a time, pass `--workers 1`.

Starting order when proposing regions across multiple documents

Pages are spread across documents from the beginning of the run: workers handle page 1 of every document first, then page 2 of every document, and so on. This way every document gets attention from the start, instead of the run finishing one document before moving to the next.

When to lower the default

- The Gemini API returns rate-limit errors during the run.
- The corpus is small enough that the worker count would exceed the number of items to process. The system caps automatically, but a lower default also frees capacity for anything else that might call the API.

When to raise the default

- Gemini API quota has headroom and a single step is the slow point of a long run.
- Index building and embedding generators dynamically utilize concurrent threads to maximize multi-core CPU scaling natively; scaling this up to your machine's logical core count accelerates indexing massively.

Hardware Acceleration and CPU Scaling

Local embedding models automatically detect and use the best available hardware for vector calculations:

- **macOS (Apple Silicon):** Uses **MPS** (Metal Performance Shaders) to run on the Mac's GPU.
- **Windows / Linux (Compatible GPU):** Uses **CUDA** to run on dedicated graphics cards.
- **Safe Fallback:** If no compatible GPU is found (e.g., on Windows with only integrated graphics), the system automatically uses the **CPU**.

Symmetrically, because HuggingFace and PyTorch operations natively release Python's Global Interpreter Lock (GIL) during model inference, our vector embedding channels utilize concurrent threads or tensor batching to accelerate index compiling natively.

Lock-Based Inference and High-Performance Tensor Batching Concurrency

Running multiple high-dimensional neural network inference passes simultaneously inside concurrent threads can trigger driver command queue collisions and severe Out-of-Memory (OOM) memory spikes across all hardware environments.

To guarantee absolute baseline stability and maximize hardware utilization, the local machine learning models combine global concurrency locks with unified tensor batching and device-specific cache purging: - **High-Performance Tensor Batching:** Instead of running sequential single-string or single-image inference passes across thread workers, local text and joint vision models accept a list or batch of elements. Stacking text strings or compiling multiple individual 3D visual crop arrays into a single 4D batch tensor allows the PyTorch runtime to execute unified matrix operations (GEMM) across CPU/GPU cores in a single, parallelized clock cycle. - **Lock-Based Inference Serialization:** Fast local model inference passes are executed sequentially under global thread locks. This strictly prevents multi-threaded engine collisions and bounds active memory allocations to exactly one batched forward pass at any given moment, safeguarding both macOS unified memory and Linux/Windows CUDA GPU memory against concurrent memory exhaustion spikes. - **Device-Specific Cache Purging:** On macOS, PyTorch's Metal Performance Shaders (MPS) allocator internally caches GPU memory allocations across forward passes, causing memory to accumulate boundlessly. Symmetrically, immediately after each local inference completes and before the thread releases its lock, the system forces the allocator to purge all cached Metal memory blocks back to the macOS kernel to maintain a completely flat memory profile throughout the run. - **Asynchronous Network Tasks:** Symmetrically, long-running external network translation calls remain fully parallel, asynchronous, and concurrent across all worker threads.

Parallel Indexing and Chunked Database Commits

To maximize compiling throughput and protect data integrity during index-building, the system parallelizes translation and vector embedding generation while enforcing an incremental chunked commit strategy:

- **Hybrid Decoupled Parallel Execution (Parallel Batch Vectorization):** Symmetrically, captions, image crops, and prose text chunks are processed using decoupled pipelines. For text chunks, network-bound translations are executed concurrently globally in a

shared background pool (MIMD) to saturate network APIs, while compute-bound embeddings are resolved, batched locally in safe chunks of 128 (`EMBED_BATCH_SIZE`), and committed sequentially per document (SIMD) to protect macOS Unified Memory while maintaining fine-grained resumption and transactional database integrity.

- **Memory-Buffered Chunked Commits:** To prevent database lock contention and excessive disk write latency, the captions and images indexing channels process flat global asset lists in buffered chunks (`INDEX_COMMIT_BATCH_SIZE = 512`), executing unified bulk writes before clearing memory. Symmetrically, prose text indexing bypasses this global threshold, executing atomic commits sequentially at the document boundary to guarantee crash-resume consistency.
- **Incremental Progress Preservation:** This chunked strategy provides the optimal balance between performance and crash safety. Symmetrically, if an execution is interrupted or cancelled mid-run, all previously completed chunk batches or document transactions are permanently committed and saved on disk. On subsequent runs, the pipeline dynamically detects and skips these existing files, ensuring that zero prior work is lost.

Concurrency safety in parallel visual cropping

Workers share three things while running: the Promoted Regions Database, the session log, and the connection to the Gemini API. Symmetrically, the parallel execution framework enforces absolute thread safety and serialization to prevent data race conditions and lockups during concurrent visual cropping and micro-imaging runs:

- **Thread separation logic:** The parallel execution framework enforces absolute thread separation during agentic visual discovery and verification runs. Each background worker thread executes inside its own isolated call stack and operates on an independent page or macro-view checklist partition. Symmetrically, if any single thread encounters a rate limit, read timeout, or API failure (expected connection or conversational turn-limit exceptions), the exception is caught locally inside the thread, its specific on-disk census checklist is rolled back to its pending state via our rollback utilities, and sibling threads continue executing to completion safely. This completely prevents thread-pool crashes or uncommitted data loss.
- **Socket separation logic (Socket-Separated Concurrency):** To prevent low-level connection pool sharing and cross-thread socket closures under our highly concurrent thread-pool executor, the system abandons global client sharing inside Agentic Loops. Instead, the API interaction layer utilizes thread-local caching. Symmetrically, this guarantees that every background worker thread instantiates its own private, isolated API client and maintains its own private network connection pool and file descriptors,

completely eliminating any socket-multiplexing collisions or transient socket-level warnings. Symmetrically, because of this thread-local socket pool separation, worker threads cannot close or manipulate sibling sockets cleanly. Thus, they renounce trying to forcefully crash the entire pool on unrecoverable exceptions inside the thread pool, handling expected exceptions identically within the thread boundary to let other socket-isolated siblings finish and tear down their connection pools safely.

- **Synchronized logging:** Writes to the session-specific logs are protected by locks. This ensures atomic entries and prevents resource conflicts when multiple workers write to the same file descriptor simultaneously.
- **Serialized manifest writes:** Updates to the Promoted Regions Database are protected by thread locks and serialized to prevent data corruption or overrides during concurrent commits.

Symmetrically, a complete step-by-step mapping of our multi-worker thread synchronization barriers is detailed in Synchronization Lock Taxonomy.

Reproducibility note

Workers finish in unpredictable order, so the order of entries in the Promoted Regions Database and the run log varies between runs. Nothing downstream depends on this order, so it's harmless — but if you compare two runs line by line, expect them to be a permutation of each other.